

Novel findings — MechanismSelector

Scope for every number: 5 streams × 4 model families, 60 runs per policy per detector (deterministic cells collapsed). ADWIN: two-sided, $\delta = 0.002$, raw 0/1 error stream. DDM: library defaults.

1. A policy's own safety guard, not its decision logic, drove half its rebuilds

What we measured: 910 of the selector's 1,825 rebuilds (49.9%) were forced by its minimum-window guard rather than by a failed nudge — 32% of all 2,830 adaptations. ADWIN, 60 runs. ([results/grid/grid.csv](#))

Why it is new: Adaptation policies are reported by accuracy and cost. We show that a precondition meant to protect the decision — refuse to judge a holdout under ~250 rows — interacts with the buffer supplying those windows and becomes the policy's dominant behaviour.

What it means: The window-supply rule is part of the policy. Clearing the buffer at every adaptation guarantees the guard fires on the next closely-following alarm.

2. A third of alarms fire on models at or above their reference accuracy — under a one-sided detector too

What we measured: With the model never changed, 2,101 of 4,680 ADWIN alarms (44.9%) and 1,720 of 4,946 DDM alarms (34.8%) fired while the model was at or above its reference accuracy. ([results/grid/events/](#), [results/grid_ddm/events/](#), [results/analysis/ddm_deficit_direction.json](#))

Why it is new: Over-sensitivity is known and usually attributed to a two-sided test reacting to improvements. A one-sided detector does it at a third of alarms, which rules that out. We report the rate only; **the mechanism is unknown.**

What it means: Treat an alarm as a request to check, not as evidence of degradation. Checking costs a holdout score; rebuilding costs a full fit.

3. Cheap repair fails differently in each model family, at the prediction level

What we measured: Per nudge, under AlwaysNudge with ADWIN: Random Forest changed **no** prediction in 60% of 1,876 nudges and Gaussian NB in 77% of 364; SGD changed a median 9.4% of predictions across 1,816 nudges for a median accuracy gain of 0.0 points; only XGBoost had a positive median gain (+2.5 points, 212 nudges). ([results/analysis/phase7.json](#), [event_split](#))

Why it is new: That cheap repair is limited is established. That the limit differs in kind — inert for bagged and closed-form learners, active but directionless for SGD — is a prediction-level measurement the accuracy-only literature does not make.

What it means: [partial_fit](#) and [warm_start](#) are not one capability. Check whether the update moves predictions at all before building a policy on it.

4. Nudging a booster moves cost from training to inference, permanently

What we measured: AlwaysNudge with XGBoost grew 100 → 365 boosting rounds on elec2 (53 nudges) and 100 → 790 on covtype (138 nudges), raising measured per-prediction node visits 2.87× and 3.18×.

([results/grid/grid.csv](#))

Why it is new: Retraining-energy studies count training. Boosting nudges transfer cost to every later prediction for the life of the model, which the work we compare against does not measure.

What it means: Report inference cost beside training cost, or a "cheap adaptation" saving is incomplete.

What we do not claim

Our method does not win: less accurate than always rebuilding (median -1.3 points, Holm $p = 0.017$), and not reliably cheaper than a fixed nudge–nudge–rebuild schedule. No policy wins on accuracy under either detector, and comparisons are detector-dependent — "FixedSchedule is cheaper" holds under ADWIN at $n = 60$, but not at $n = 20$ and not under DDM. Five streams is a small sample, and cells within a stream share drift points. Finding 2 has no established mechanism. Finding 1 describes this implementation's defaults, not every guarded policy.